

Arquitetura de GPUs

André Luís Mendes Júnior

Bruna Fontes Castro

Felipe do Amparo Lauria

Instituto de Biociências, Letras e Ciências Exatas, Unesp -
Universidade Estadual Paulista, 15054-000, São José do Rio
Preto - SP,
Brazil.

April 2024

1 Introdução

Até a década de 1980, os pesquisadores em computação enfrentavam significativos desafios ao lidarem com operações longas e altamente ramificadas. As CPUs (Central Processing Units), que eram os principais componentes de processamento dos computadores, apresentavam limitações físicas e processuais que dificultavam a execução eficiente de tarefas complexas. Essas limitações eram particularmente evidentes em cálculos intensivos e gráficos avançados, onde a capacidade de paralelização era crucial.

Nesse contexto, a criação das GPUs (Graphics Processing Units) marcou um avanço revolucionário. Diferentemente das CPUs, que são projetadas para lidar com um número menor de tarefas em série de maneira extremamente rápida, as GPUs são arquitetadas para processar milhares de operações simultaneamente. Essa capacidade de paralelização massiva permite que as GPUs executem cálculos complexos e gráficos detalhados com uma eficiência muito superior às CPUs.

Inicialmente desenvolvidas para acelerar a renderização de gráficos em computadores pessoais e consoles de videogame, as GPUs rapidamente mostraram seu valor em uma ampla gama de aplicações além dos gráficos. Sua

arquitetura paralela as tornou ideais para tarefas de processamento intensivo, como simulações científicas, aprendizado de máquina e outras formas de computação de alto desempenho.

A evolução das GPUs transformou não apenas a forma como os gráficos são processados, mas também abriu novas fronteiras na ciência da computação, permitindo avanços em áreas que exigem grande capacidade de processamento paralelo. Assim, enquanto as CPUs continuam sendo fundamentais para a execução de operações sequenciais e tarefas gerais, as GPUs desempenham um papel crucial em permitir que as máquinas lidem com cargas de trabalho complexas e intensivas de maneira eficiente e rápida.

2 História

Origens e Primeiras GPUs

As GPUs tiveram suas raízes nos primeiros aceleradores gráficos, que eram componentes dedicados à renderização de gráficos em sistemas de computação. Na década de 1980, empresas como a IBM começaram a incluir chips gráficos em seus sistemas, mas esses componentes eram limitados em capacidade e funcionalidades. Foi apenas na década de 1990 que as GPUs começaram a se diferenciar como unidades de processamento distintas, com a introdução do chip S3 86C911 pela S3 Graphics em 1991, que trouxe capacidades aprimoradas para o processamento gráfico 2D.

A verdadeira transformação começou com a introdução da série Voodoo Graphics pela 3dfx Interactive em 1996. Esses chips eram projetados especificamente para acelerar a renderização de gráficos 3D, oferecendo desempenho significativamente superior ao dos processadores de propósito geral. A Voodoo Graphics estabeleceu as bases para a arquitetura de GPUs dedicadas ao processamento paralelo massivo de dados gráficos.

Evolução das Arquiteturas de GPUs

Primeira Geração: Fixed-Function Pipeline

Nos anos 90 e início dos anos 2000, as GPUs seguiam a arquitetura de pipeline de funções fixas (Fixed-Function Pipeline). Esse modelo era caracterizado por um fluxo de processamento gráfico predeterminado, onde cada estágio do pipeline (transformação, iluminação, configuração, rasterização) era implementado diretamente no hardware. Exemplos notáveis incluem a série GeForce 256 da NVIDIA, lançada em 1999, e as ATI Radeon R100, lançada em 2000.

Essas GPUs eram eficazes para gráficos 3D, mas tinham flexibilidade limitada. Cada estágio do pipeline tinha funcionalidades predefinidas, e os desenvolvedores de software não podiam modificar diretamente esses estágios para operações específicas.

Segunda Geração: Programmable Pipeline

A introdução da NVIDIA GeForce 3 em 2001 marcou o início das GPUs com pipeline programável. A arquitetura de pipeline programável permitia que os desenvolvedores utilizassem shaders programáveis, possibilitando maior flexibilidade e controle sobre o processo de renderização. Os Vertex Shaders e Pixel Shaders permitiam cálculos personalizados em vértices e pixels, respectivamente, permitindo gráficos mais complexos e detalhados.

A ATI (agora AMD) também seguiu essa tendência com suas séries Radeon 8000 e posteriores, como a Radeon 8500, lançada em 2001, oferecendo capacidades semelhantes. Essas arquiteturas foram vitais para o desenvolvimento de gráficos realistas e abriram caminho para usos não gráficos das GPUs, como em cálculos científicos e criptografia.

Terceira Geração: Unified Shader Model

A partir de meados dos anos 2000, as GPUs começaram a adotar o modelo de shader unificado, onde um único tipo de unidade de processamento (shader) podia realizar tanto operações de vértice quanto de pixel. A NVIDIA introduziu esta arquitetura com a série GeForce 8800 em 2006. Este modelo permitia uma melhor utilização dos recursos de hardware, pois as unidades de shader podiam ser alocadas dinamicamente para diferentes tipos de tarefas conforme necessário.

As GPUs da ATI Radeon HD 2000, lançadas em 2007, também implementaram esta arquitetura, permitindo não apenas gráficos mais avançados, mas também facilitando o uso de GPUs para computação geral (GPGPU) através de APIs como CUDA (NVIDIA) e OpenCL (Khronos Group).

Arquiteturas Modernas: Heterogeneous Systems e Deep Learning

Nas últimas décadas, a evolução das GPUs tem sido impulsionada pela demanda crescente por poder de processamento paralelo em aplicações como aprendizado de máquina e inteligência artificial. A NVIDIA introduziu a arquitetura CUDA (Compute Unified Device Architecture) em 2006, que transformou GPUs em plataformas poderosas para computação científica e técnicas de aprendizado profundo.

A arquitetura Pascal, introduzida em 2016, e posteriormente a arquitetura Volta, em 2017, foram marcos importantes, trazendo unidades "tensor cores" dedicadas para operações de aprendizado profundo. A arquitetura Turing, lançada em 2018, e a Ampere, em 2020, continuaram essa tendência, com melhorias significativas em desempenho e eficiência energética.

AMD, por sua vez, evoluiu suas arquiteturas com a série Radeon Vega e, mais recentemente, com as arquiteturas RDNA e RDNA 2, lançadas em 2019 e 2020 respectivamente, que competem diretamente com as ofertas da NVIDIA em termos de desempenho e eficiência.

3 Hardware

As GPUs foram projetadas para executar diversos tipos de operações gráficas. Geralmente, nas aplicações gráficas, uma mesma operação é repetida inúmeras vezes com diferentes dados, o que levou ao desenvolvimento de uma arquitetura de GPU que opera grandes conjuntos de instruções de maneira paralela. Para tornar esse paralelismo eficiente, o hardware foi implementado com base em pipelines gráficos. Esses pipelines foram implementados em várias unidades independentes, permitindo que as GPUs mantenham altas frequências de computação em execuções paralelas de instruções.

Os pipelines gráficos são estruturas fundamentais nas GPUs, dividindo o processo de renderização em várias etapas sequenciais, como transformação de vértices, mapeamento de texturas e rasterização. Cada etapa do pipeline é processada por diferentes unidades de hardware, o que permite que múltiplas etapas sejam executadas simultaneamente em dados diferentes, aumentando significativamente a eficiência e a velocidade da computação gráfica.

Além disso, as GPUs exploram dois tipos principais de paralelismo: paralelismo a nível de thread e paralelismo de dados. O paralelismo a nível de thread é obtido através da execução simultânea de milhares de threads em múltiplos núcleos de processamento. Cada thread pode executar uma instrução diferente em diferentes conjuntos de dados, permitindo que as GPUs processem uma grande quantidade de dados em paralelo.

O paralelismo de dados, por outro lado, envolve a aplicação da mesma instrução a vários dados simultaneamente. Isso é especialmente útil em operações como multiplicações de matrizes e transformações de vértices, onde a mesma operação é aplicada repetidamente a diferentes elementos de dados. As GPUs são projetadas com uma arquitetura SIMD (Single Instruction, Multiple Data), que permite que uma única instrução seja aplicada a vários dados ao mesmo tempo, maximizando a eficiência do processamento.

Outra característica importante das GPUs é sua especialização, resul-

tando em uma eficiência significativamente maior em comparação com hardwares de propósito geral, como as CPUs. Com o passar dos anos, a diferença de performance entre GPUs e CPUs aumentou, como mostra o gráfico da figura 1. Uma segunda vantagem das GPUs sobre as CPUs é que a maioria de seus transistores é dedicada à computação, enquanto apenas uma pequena parte é destinada ao controle. Isso resulta em um maior poder de computação, mas também limita um pouco os fluxos de programas.

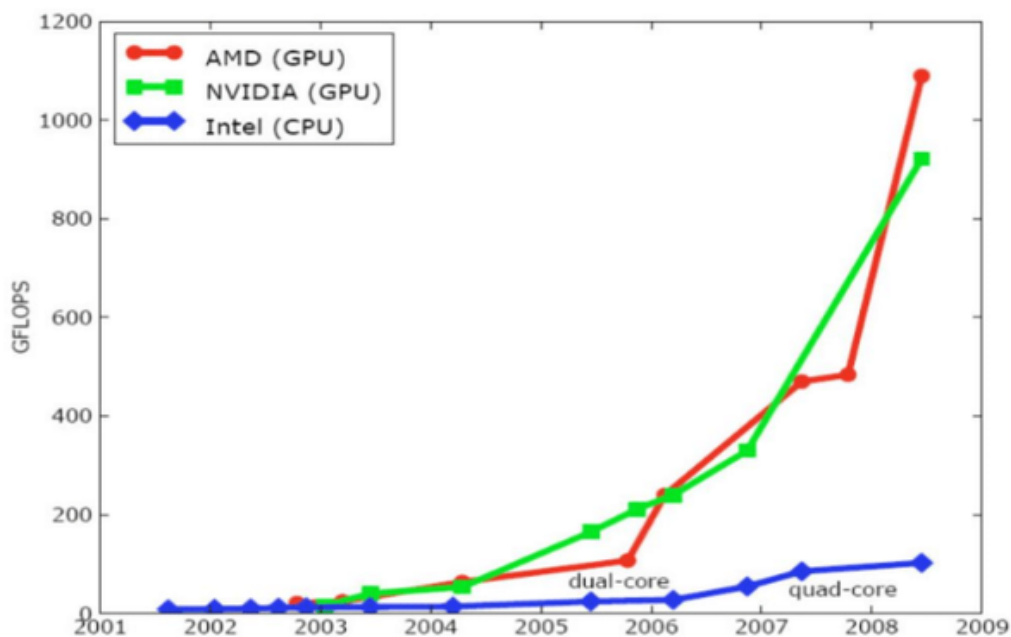


Figura 1: Evolução da capacidade de processamento das placas de vídeo GeForce e ATI comparados com os processadores Intel.

Além disso, no acesso à memória, as GPUs maximizam o throughput, enquanto as CPUs priorizam a latência. Isso ocorre porque as GPUs aproveitam melhor o princípio da localidade em comparação com a computação genérica, valorizando mais a eficiência na transferência de um conjunto de elementos do que o tempo gasto para acessá-los. Atualmente, a GPU é o tipo de hardware mais acessível para computação de alto desempenho quando a tarefa a ser processada se beneficia de seu alto grau de paralelismo. No entanto, apesar de as GPUs estarem se tornando cada vez mais programáveis, ainda são necessários vários fatores para alcançar a melhor performance possível para uma aplicação específica.

Dada essa eficiência das GPUs, pode-se imaginar que uma aplicação executada em uma CPU teria melhor performance ao ser executada em uma

GPU. Contudo, essa é uma concepção simplista, pois nem toda aplicação pode ser diretamente adaptada para uso em uma GPU. É necessário remodelar o problema para um formato gráfico, o que nem sempre é possível na maioria dos casos.

4 Modelo de Programação

As constantes evoluções no hardware das GPUs ao longo dos anos permitiram que ela se tornasse altamente programável, uma vez que houve foco em sua arquitetura na parte de das ULAs (Unidade Lógica e Aritmética), trazendo grande capacidade aritmética, não restringindo a GPU às operações gráficas. Atualmente, as GPUs realizam processamento massivo paralelo e sua programabilidade e desempenho em cálculos que exigem um volume grande de dados, resultam em um grande throughput, diferindo das tarefas sequenciais e de baixa latência da CPU.

As GPUs seguem o paradigma de execução Single Instruction Multiple Data (SIMD) - ou, em processadores da NVIDIA mais recentes, o Single Instruction Multiple Threads (SIMT) o que significa que a mesma instrução é executada em várias threads diferentes- ideal para operações com vetores ou matrizes, além da renderização e cálculos com ponto flutuante. Primordialmente, os programadores faziam códigos para a programação paralela usando APIs como o OpenGL e DirectX utilizando coordenadas, shaders, texturas e cálculos para renderização, o que acabava por ser um processo extremamente complexo, trabalhoso, suscetível a falhas e exigia muito conhecimento de arquitetura por parte do desenvolvedor.

A partir disso, como não era viável pedir aos pesquisadores para lidar tanto com restrições severas de recursos e programação quanto para aprender gráficos de computador e linguagens e comandos específicos, a NVIDIA trouxe uma nova arquitetura chamada Compute Unified Device Architecture (CUDA). Essa arquitetura incluía vários novos componentes projetados estritamente para computação com GPU, removendo limitações do processador, o levando a servir para a computação de propósito geral. Uma de suas principais características foram as mudanças nas ULAs, que melhoraram a aritmética de ponto flutuante de precisão simples e foram projetadas para usar um conjunto de instruções voltado para computação geral, em vez de especificamente para gráficos. Além disso, a NVIDIA forneceu um driver de hardware que facilitava a programação, e também desenvolveu a linguagem CUDA C, baseada em C/C++, a primeira linguagem projetada especificamente por uma empresa de GPU para facilitar a computação de propósito geral em GPUs.

As GPUs são organizadas em uma hierarquia de unidades de processamento, com núcleos agrupados em unidades de multiprocessamento. Cada multiprocessador pode executar centenas de threads simultaneamente, organizados em blocos de threads. Em modelos de programação como CUDA (da NVIDIA) ou OpenCL (um padrão aberto), um programa, chamado de kernel, é escrito para ser executado por cada thread. O programador precisa definir como os dados serão distribuídos entre esses threads e como as threads serão organizadas em blocos e grades.

A programação de GPUs requer uma consideração cuidadosa do gerenciamento de memória. Existem diferentes tipos de memória na GPU, incluindo memória global, memória compartilhada e registradores, cada uma com suas características de velocidade e acessibilidade. A memória global é grande, mas relativamente lenta, enquanto a memória compartilhada e os registradores são muito mais rápidos, mas limitados em tamanho. Os programadores precisam otimizar o uso dessas diferentes memórias para minimizar latências e maximizar o desempenho. Transferências de dados entre a CPU e a GPU também devem ser gerenciadas eficientemente para evitar gargalos.

A sincronização entre threads é crucial na programação de GPUs para evitar condições de corrida e garantir a correção dos resultados. Em CUDA, por exemplo, existem instruções de sincronização como `__syncthreads()` que garantem que todas as threads em um bloco alcancem um certo ponto antes de continuar. No entanto, a comunicação e sincronização entre blocos de threads são limitadas, o que requer que os algoritmos sejam projetados para minimizar a dependência entre blocos, além do uso de operações atômicas que garantem a consistência dos dados em acessos concorrentes.

5 Aplicações

As placas de vídeo, ou GPUs, representam componentes tecnológicos fundamentais com uma gama diversificada de aplicações que abrangem diversos setores industriais e campos de pesquisa. Este texto acadêmico tem como objetivo discutir detalhadamente as aplicações das GPUs em diferentes áreas, destacando sua importância e contribuição para avanços significativos em tecnologia e ciência.

Na aviação, as GPUs desempenham um papel crítico no processamento de dados provenientes de imagens e sensores instalados em aeronaves. Esses dados são essenciais para garantir a segurança e a eficiência das operações aéreas, incluindo a navegação precisa e a detecção de possíveis falhas técnicas. A capacidade das GPUs de processar grandes volumes de dados em tempo

real é fundamental para garantir uma resposta rápida a eventos adversos e para otimizar o desempenho geral das aeronaves.

Na indústria automotiva, as GPUs são utilizadas em sistemas de assistência ao condutor e em veículos autônomos ou semiautônomos. Elas desempenham um papel crucial no processamento de dados provenientes de uma variedade de sensores, como câmeras, radares e LiDAR, para detectar e interpretar o ambiente ao redor do veículo. Essa capacidade de processamento é vital para orientar as operações dos veículos, desde a detecção de obstáculos até a navegação em estradas complexas e variadas.

No campo da saúde, as GPUs são empregadas na interpretação e análise de imagens médicas em exames diagnósticos, como tomografias computadorizadas (CT), ressonâncias magnéticas (MRI) e ultrassonografias. O processamento rápido e preciso dessas imagens é crucial para auxiliar os médicos no diagnóstico e no tratamento de uma variedade de condições médicas, desde doenças cardíacas até cânceres. Além disso, as GPUs também são utilizadas em simulações computacionais para modelagem de órgãos e processos biológicos, contribuindo para avanços significativos na pesquisa médica e no desenvolvimento de novas terapias.

Na indústria dos jogos, as GPUs têm sua aplicação mais destacada, sendo essenciais para criar ambientes virtuais imersivos e garantir a execução suave de jogos complexos e exigentes em termos de recursos gráficos. A demanda por capacidade de processamento gráfico na indústria dos jogos impulsiona continuamente o desenvolvimento de tecnologias GPU mais avançadas e eficientes, proporcionando experiências de jogo cada vez mais realistas e envolventes.

Além disso, as GPUs desempenham um papel significativo nas pesquisas acadêmicas, especialmente na área de Inteligência Artificial (IA). Sua capacidade de executar múltiplos processos simultaneamente é crucial para lidar com as demandas computacionais intensivas desses projetos de IA, incluindo o treinamento de modelos de aprendizado profundo e a execução de algoritmos de processamento de linguagem natural. As GPUs oferecem uma solução viável para essa necessidade, proporcionando poder de processamento massivamente paralelo que pode rivalizar com o desempenho de supercomputadores em muitos casos.

Em síntese, as GPUs desempenham um papel multifacetado e essencial em uma variedade de aplicações, desde a aviação até a saúde, passando pela indústria dos jogos e pela pesquisa acadêmica em IA. Seu papel crítico no processamento de dados e na execução de tarefas computacionais intensivas tem impulsionado avanços significativos em diversas áreas, contribuindo para o desenvolvimento tecnológico e científico em escala global.

6 Futuro

O primeiro desafio a ser considerado diz respeito às tendências da tecnologia. Cada nova geração de hardware apresenta um novo desafio para os fabricantes de GPUs, pois é necessário integrar novos recursos de hardware de maneira eficiente, com o objetivo de melhorar a performance e a funcionalidade. Um número crescente de transistores é adicionado às placas gráficas, permitindo maior exploração de paralelismo e novas funcionalidades ao pipeline gráfico.

Além disso, é crucial garantir a eficiência na comunicação. O aumento dos custos relacionados a este fator influencia as microarquiteturas dos futuros chips. Portanto, os desenvolvedores devem planejar cuidadosamente o tempo necessário para o envio de dados ao longo dos chips.

Outro desafio significativo é o gargalo das GPUs, que representa as limitações que impedem a GPU de atingir seu desempenho máximo. Um dos principais gargalos é a largura de banda da memória. À medida que as GPUs se tornam mais poderosas, a quantidade de dados que precisam ser transferidos entre a memória e os núcleos de processamento aumenta substancialmente. Se a largura de banda da memória não acompanhar essa demanda, a GPU não poderá operar em seu potencial máximo, resultando em um desempenho subótimo. Melhorar a largura de banda da memória, seja por meio de novas tecnologias de memória, como HBM (High Bandwidth Memory), ou através de arquiteturas inovadoras, será essencial para superar esse desafio.

Outro gargalo importante é a comunicação entre componentes dentro da própria GPU e entre a GPU e a CPU. À medida que mais transistores são adicionados para aumentar o paralelismo e a funcionalidade, a complexidade da comunicação também aumenta. As latências na transferência de dados podem se tornar um impedimento significativo para a eficiência geral do sistema. Soluções para esse problema incluem o desenvolvimento de interconexões mais rápidas e eficientes, bem como a otimização das arquiteturas de comunicação interna.

Além disso, o consumo de energia é um fator que tem se tornado cada vez mais crítico no desenvolvimento das GPUs modernas, uma vez que cada nova geração de hardware demanda mais energia. A demanda crescente por desempenho leva a um aumento no consumo de energia, o que por sua vez gera mais calor e requer soluções de resfriamento mais eficazes. Espera-se, portanto, um maior gerenciamento de energia nos estágios do pipeline, bem como sistemas de resfriamento ainda mais sofisticados. Novas técnicas de gerenciamento de energia, como a redução dinâmica da voltagem e da frequência, e sistemas de resfriamento mais avançados, serão necessários para enfrentar esses desafios.

Por fim, temos a questão da programabilidade e das funcionalidades oferecidas pelas GPUs. Mesmo com todo o progresso alcançado até hoje, a programabilidade em GPUs ainda está longe do ideal. Uma ideia para a evolução destes fatores é melhorar a flexibilidade e as funcionalidades das unidades programáveis atuais (de vértices e de fragmentos). Possivelmente, seus conjuntos de instruções podem convergir, tornando seus controles de fluxo mais generalizados. Podemos também observar um compartilhamento de hardware programável entre esses dois estágios, a fim de utilizar melhor seus recursos. Outra opção é expandir a programabilidade para unidades diferentes.

7 Conclusão

A evolução da arquitetura das GPUs desde suas origens como aceleradoras gráficas até sua posição central na computação moderna é marcada por avanços significativos em programabilidade e eficiência. Inicialmente focadas em gráficos, as GPUs agora suportam uma ampla gama de aplicações, como aprendizado de máquina e simulações científicas, graças ao desenvolvimento de unidades de processamento paralelo e memórias otimizadas. Plataformas como CUDA e OpenCL democratizaram o acesso à potência de processamento massivo das GPUs. No futuro, espera-se que as GPUs continuem evoluindo para suportar tecnologias emergentes como inteligência artificial e computação quântica, mantendo seu papel crucial na inovação tecnológica e atendendo à crescente demanda por desempenho computacional avançado.

Referências

- [1] Jason Sanders, Edward Kandrot . **CUDA by Example: An Introduction to General-Purpose GPU Programming**. Addison-Wesley, 2010.
- [2] Corporation, NVidia. **The World's First GPU: GeForce 256**. NVidia Corporation, 1999.
- [3] Fernando, R. **GPGPU: General-Purpose Computation on GPUs**. NVIDIA Developer Technology, 2004.
- [4] Anselmo Ferreira, Leandro Zanutto, Marcelo Matsumoto. **Arquitetura e Programação de GPU Nvidia**. Instituto de Computação Universidade Estadual de Campinas, 2012.